

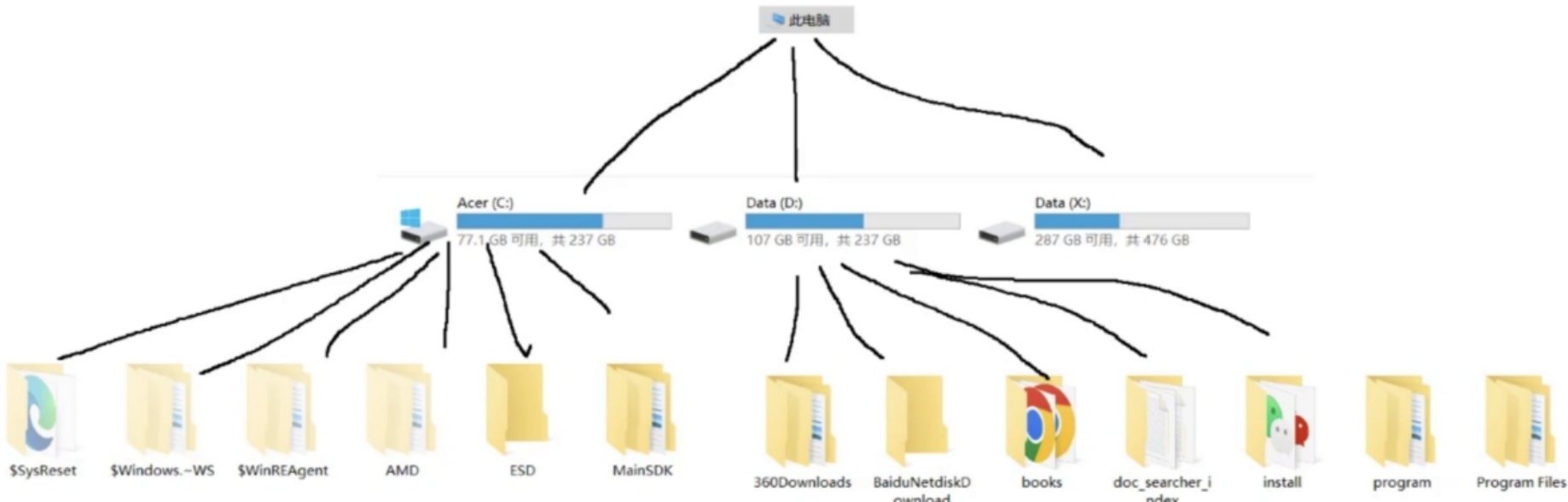
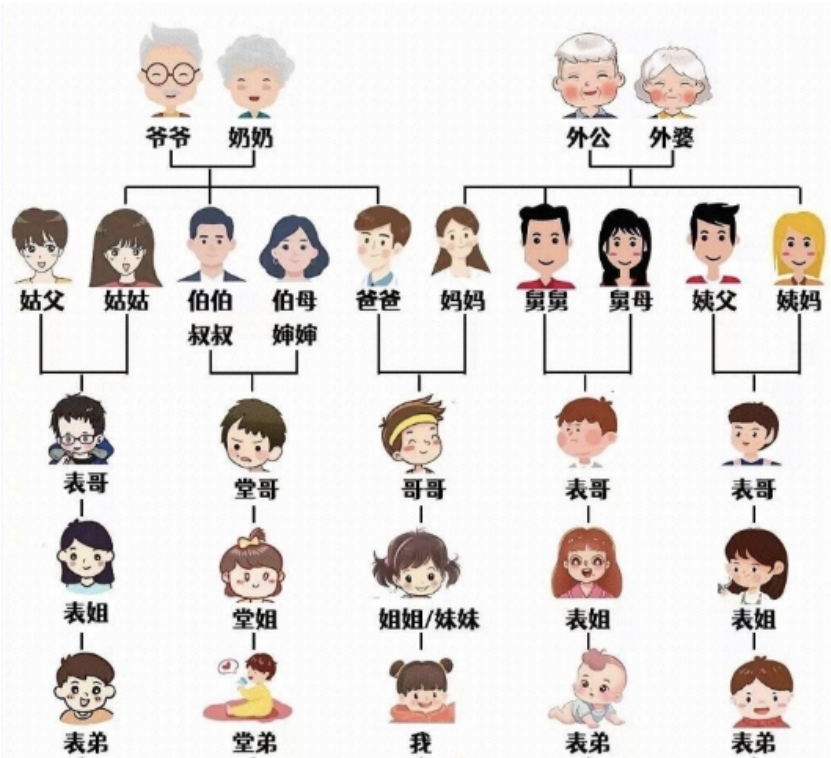
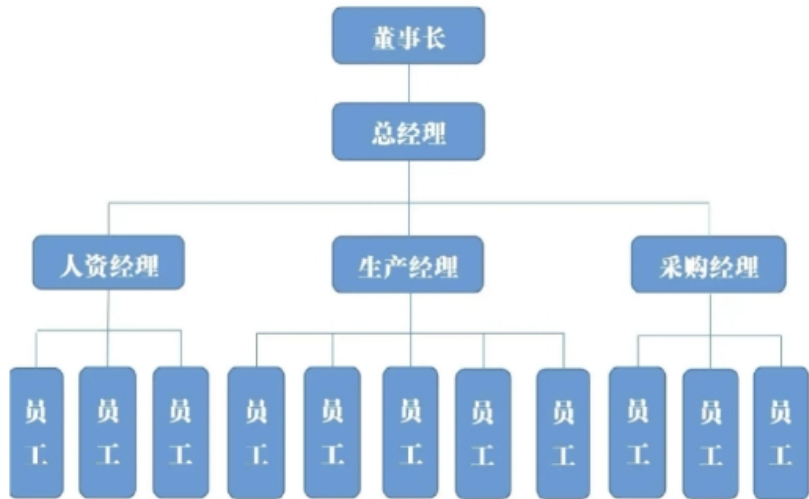
CSP-J——树/图专题

树

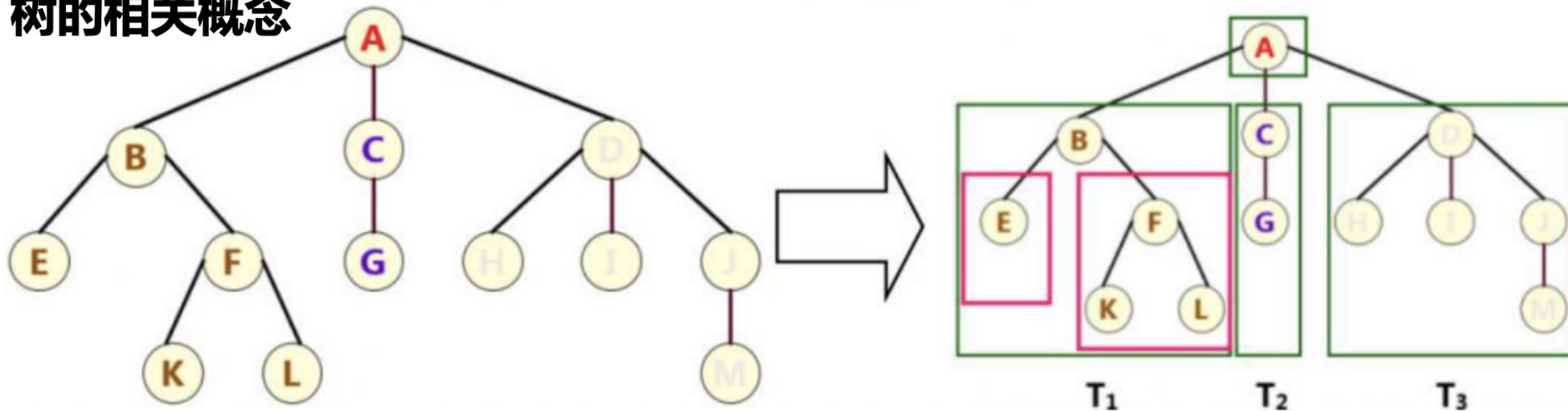
树（tree），是一种数据结构，它是由 n ($n \geq 0$)个有限节点组成一个具有层次关系的集合。树是一类非线性结构。这种结构结点之间有分支，并具有层次关系。它非常类似于自然界中的树。

树的作用：表达家谱顺序、行政组织结构、计算机文件结构等。

组织机构图

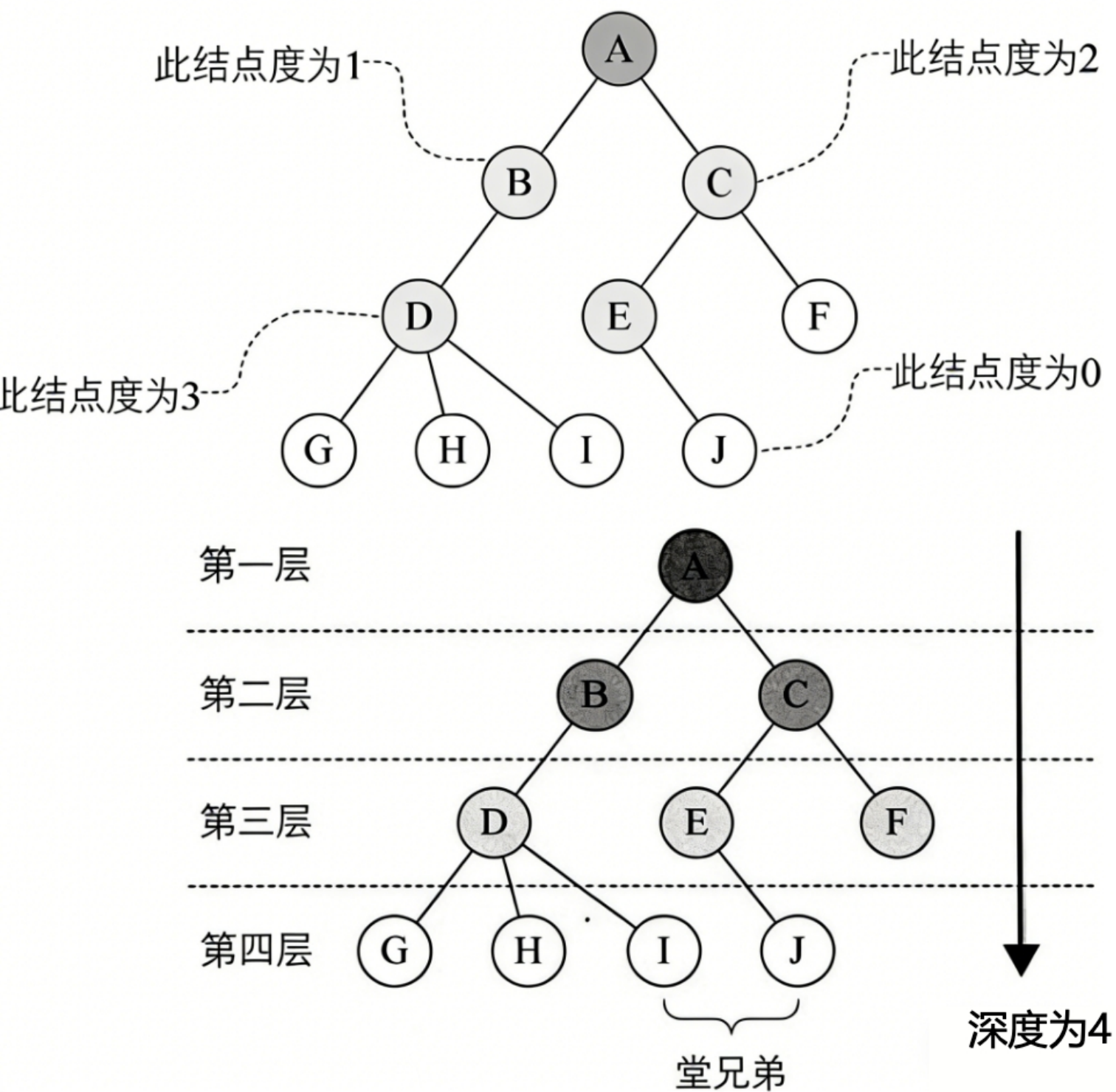


树的相关概念



1.当 $n=0$ 时称为空树

2.当 $n>0$ 时为非空树，在任意一棵非空树中，有且仅有一个**称为根的结点**，其余的结点可分为 m 个互不相交的有限集 $T_1, T_2, \dots, T_m$ ，其中每一个集合又称为一棵树，也就是根的子树；同理，每一棵子树又可以分为若干个互不相交的有限集.....



树的结点：包含一个数据元素及若干个指向其子树的分支；

结点的度：一个结点拥有的子树数目；

树的度：一棵树上所有结点度的最大值；

叶子结点(终端结点)：度为零的结点；

分支结点(非终端结点)：度大于零的结点；

路径(从根到结点的)：由从根到该结点所经分支和结点构成；

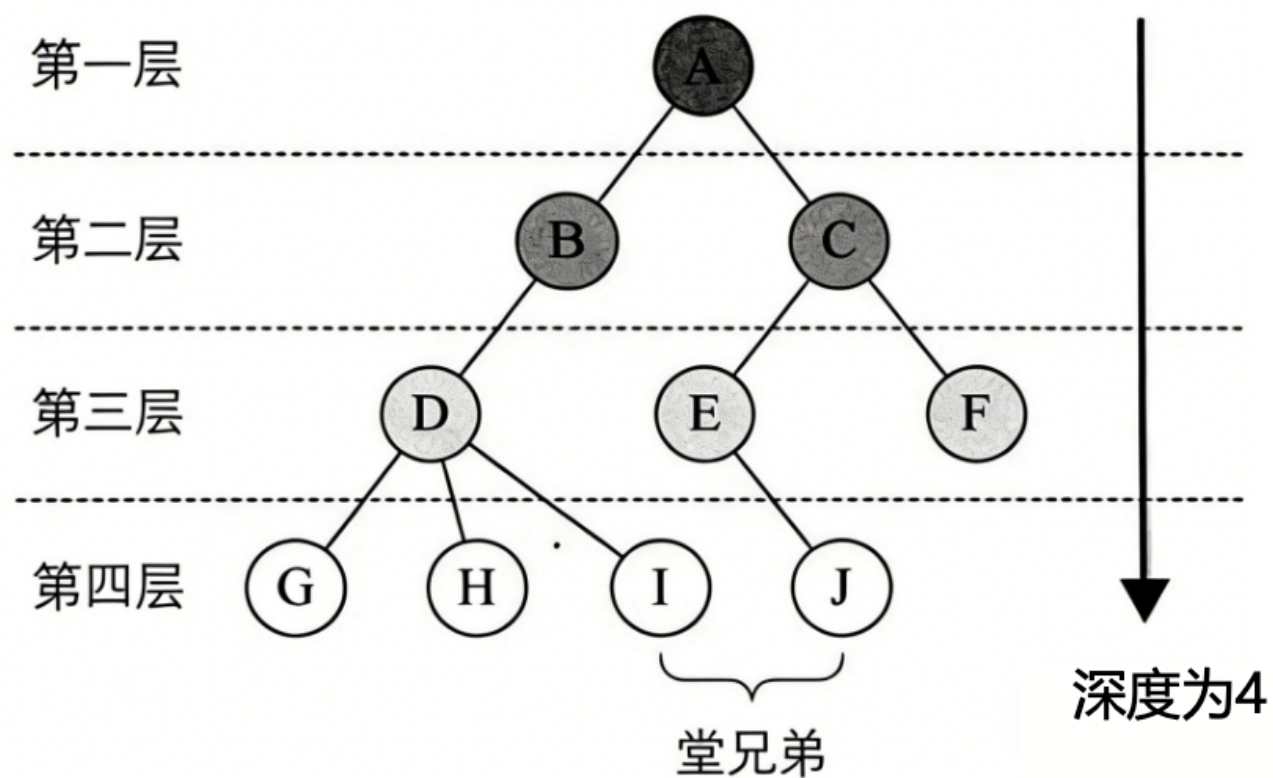
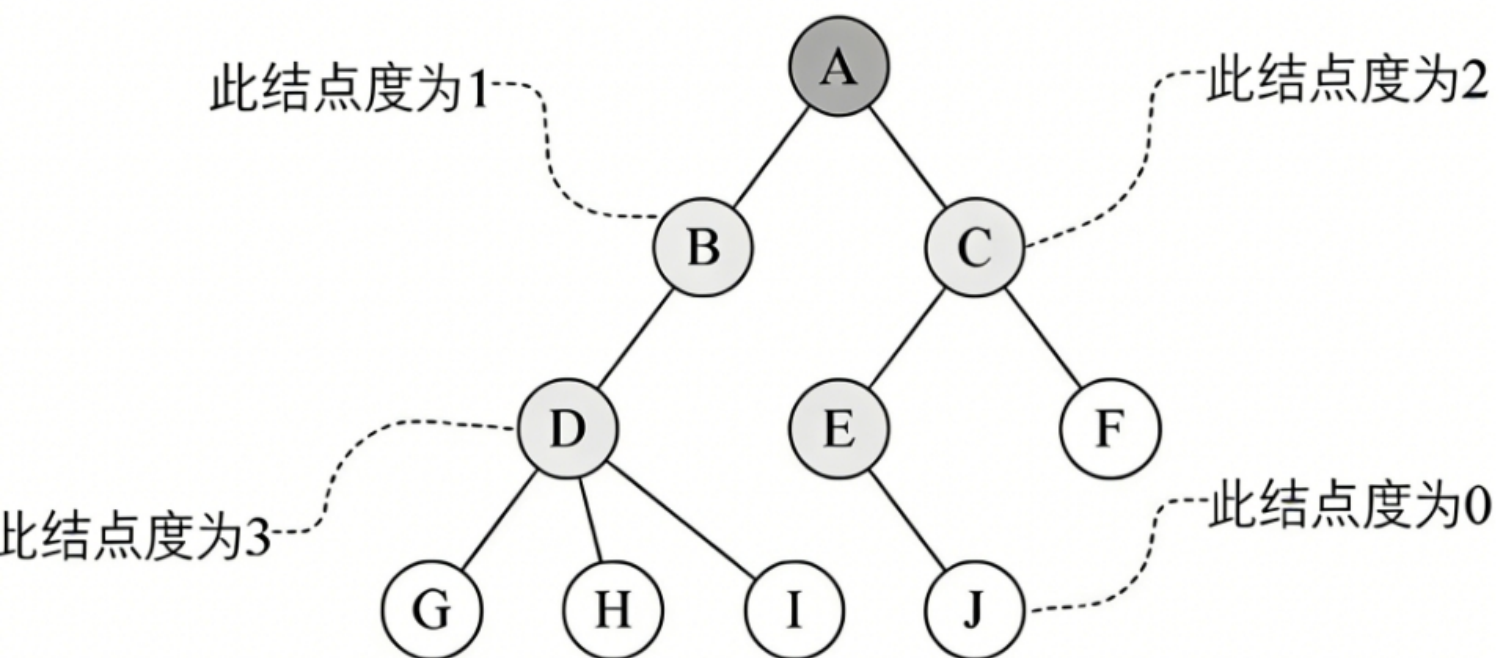
孩子结点：结点的子树的根称为该结点的孩子结点；

双亲结点：相应地，该结点称为孩子的双亲结点；

兄弟：具有同一父结点的子结点互称兄弟；

堂兄弟：其双亲在同一层的结点互为堂兄弟；

祖先结点：从根到该结点所经分支上的所有结点；



子孙结点：以某结点为根的子树中任一结点都称为该结点的子孙；

结点的层次：从根结点到该结点所经过的路径长度加1；

树的深度：树中叶子结点具有的最大层次数；

树的宽度：整棵树中某一层中最多的结点数称为树的宽度；

有序树：如果将树中结点的各子树看成从左至右是有次序的(即不能互换)，则称该树为有序树，与之相对的是无序树；

第一个孩子：在有序树中，最左边的子树的根称为第一个孩子；

最后一个孩子：在有序树中，最右边的子树的根称为最后一个孩子；

二叉树

二叉树的每个结点至多只有**二棵子树**(不存在度大于2的结点)。二叉树的子树有左子树、右子树。其次序不能颠倒。所以二叉树是一棵严格的有序树。

树的边数等于节点数-1

非空二叉树满足：度为0的叶子节点=度为2的节点+1

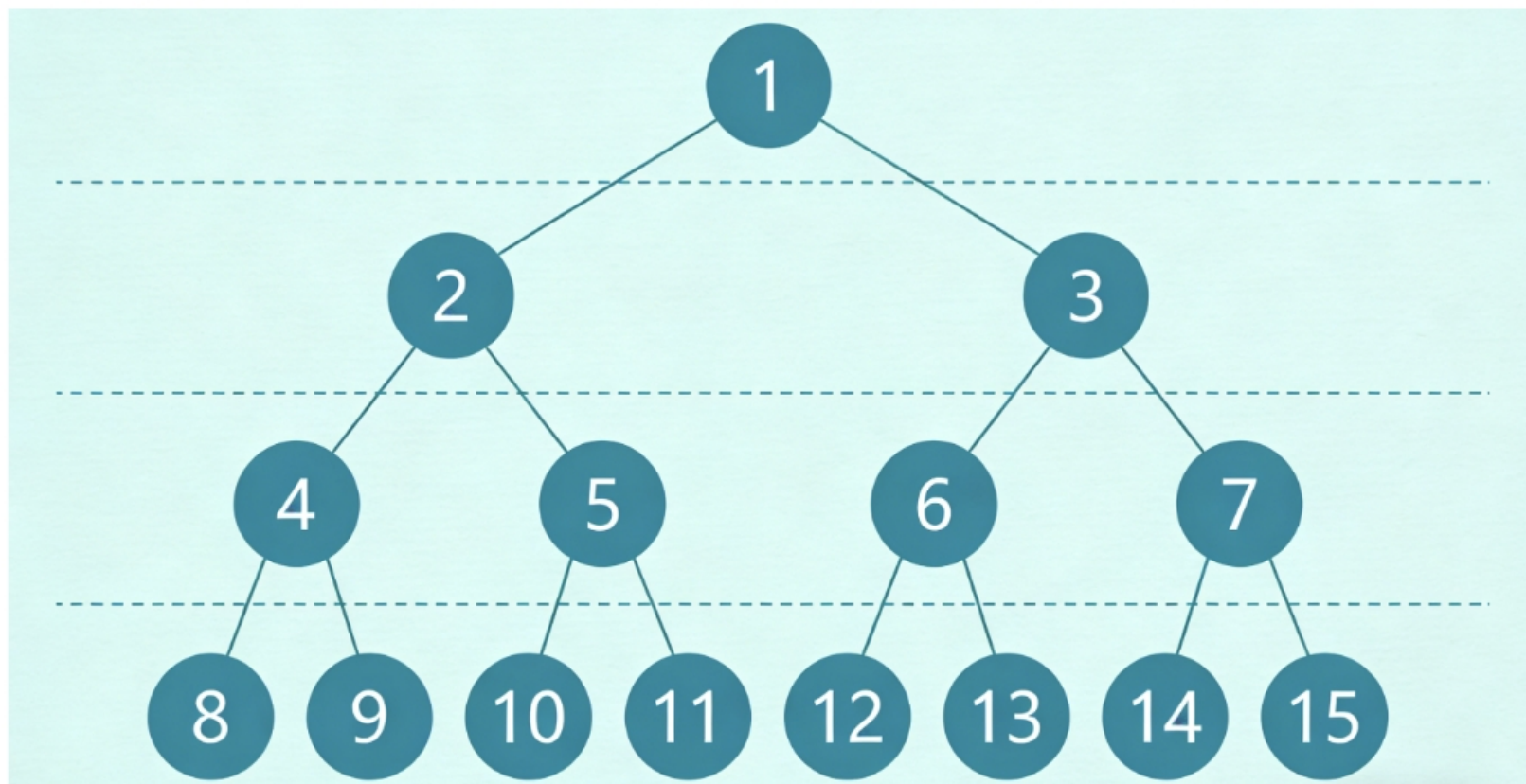
满二叉树，完美二叉树 (perfect binary tree)除最后一层无任何子结点外，每一层上的所有结点都有两个子结点。

设深度为 h ;

叶子数为 $2^{(h-1)}$;

第 k 层的结点数是： $2^{(k-1)}$

总结点数是： $(2^k)-1$ ，且总结点数一定是奇数。

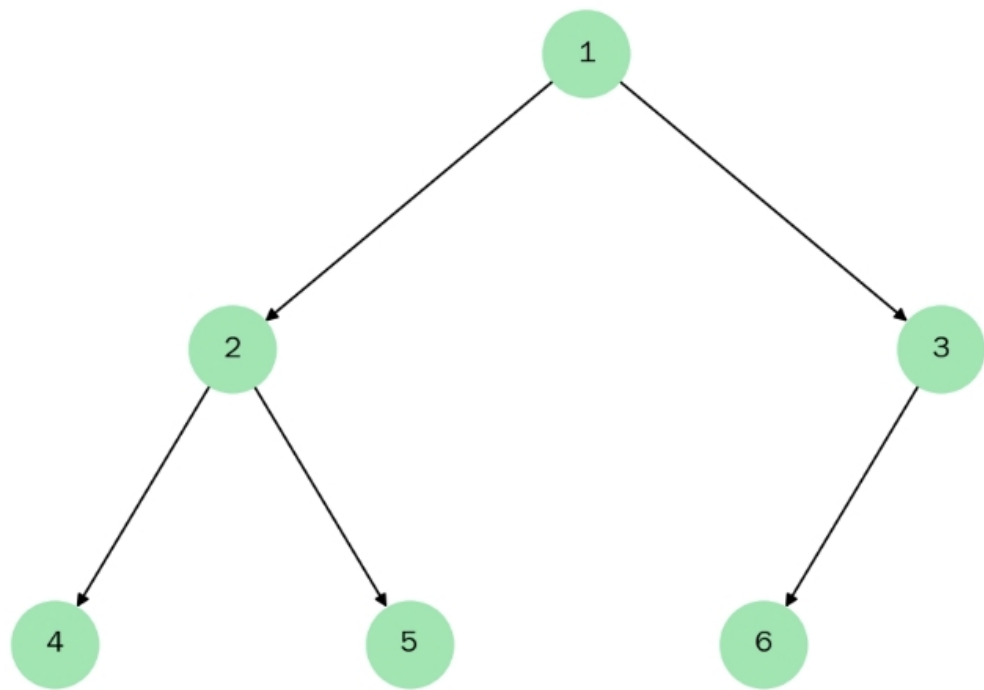


完全二叉树 (complete binary tree) 若设二叉树的深度为 h ，除第 h 层外，其它各层 ($1 \sim (h-1)$ 层) 的结点数都达到最大个数，第 h 层所有的结点都连续集中在最左边，这就是完全二叉树。

若对含 n 个结点的完全二叉树从上到下且从左至右进行 1 至 n 的编号，编号 i 的结点满足以下特性：

- 若 $i = 1$ ，则该结点是二叉树的根，无双亲，否则，编号为 $\lfloor i/2 \rfloor$ 的结点为其双亲结点；
- 若 $2i > n$ ，则该结点无左孩子，否则，编号为 $2i$ 的结点为其左孩子结点；
- 若 $2i+1 > n$ ，则该结点无右孩子结点，否则，编号为 $2i+1$ 的结点为其右孩子结点。

完全二叉树（最后一层靠左、可不满）

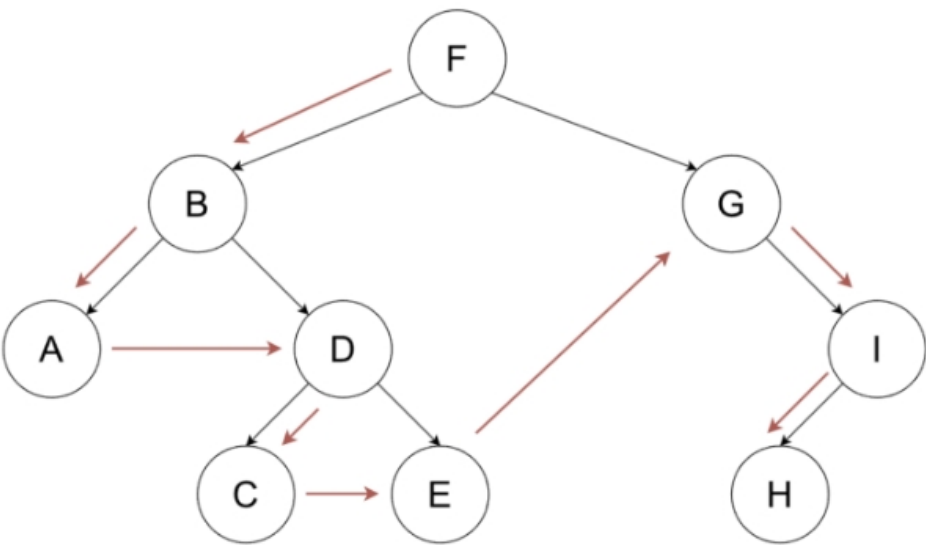


斜树：所有节点都只有左子树（左斜树）或都只有右子树（右斜树） （链表）

二叉搜索树：每个节点的左子树所有节点的值小于当前节点的值，右子树所有节点的值大于当前节点的值。

平衡二叉树：任意节点的左右子树高度差的绝对值 ≤ 1

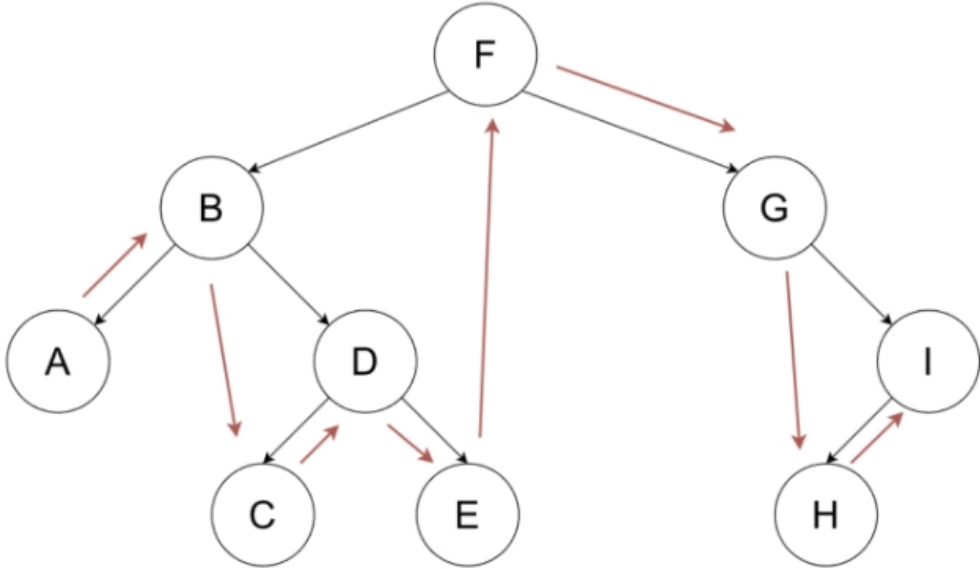
二叉树的DFS（深度优先）遍历



前序遍历

FBADCEGIH

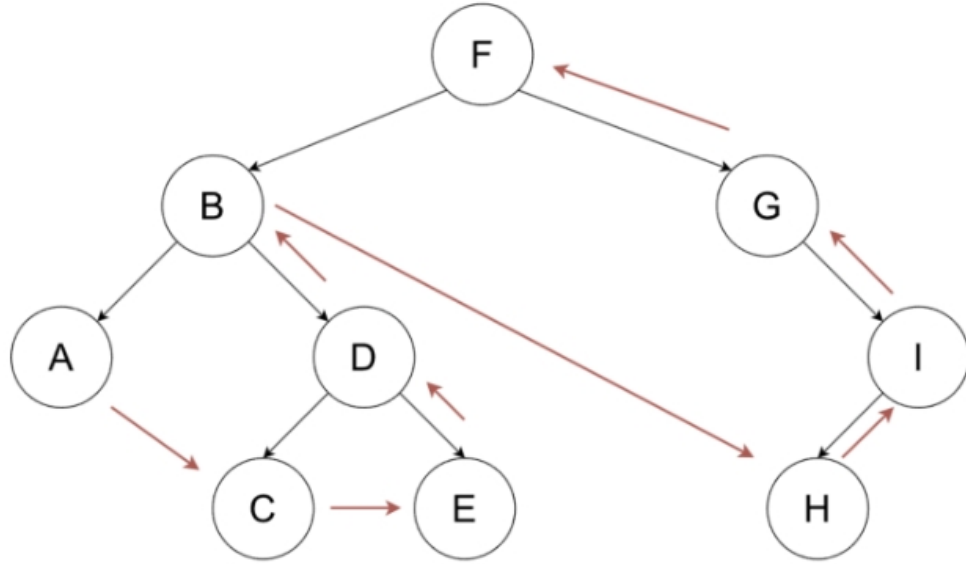
根—左子树—右子树



中序遍历

ABCDEFIGH

左子树—根—右子树



后序遍历

ACEDBHIGHF

左子树—右子树—根

小练习.已知前序和中序, 求后序

前序: ABDECFG

中序: DEBACFG

选项:

A. EDBGFCA

B. EDBGCF A

C. DEBGFCA

D. DBEGFCA

答案:



- 1.找出根节点, 分两边
- 2.继续上述步骤对左子树和右子树进行操作
- 3.画出树, 检验结果

技巧

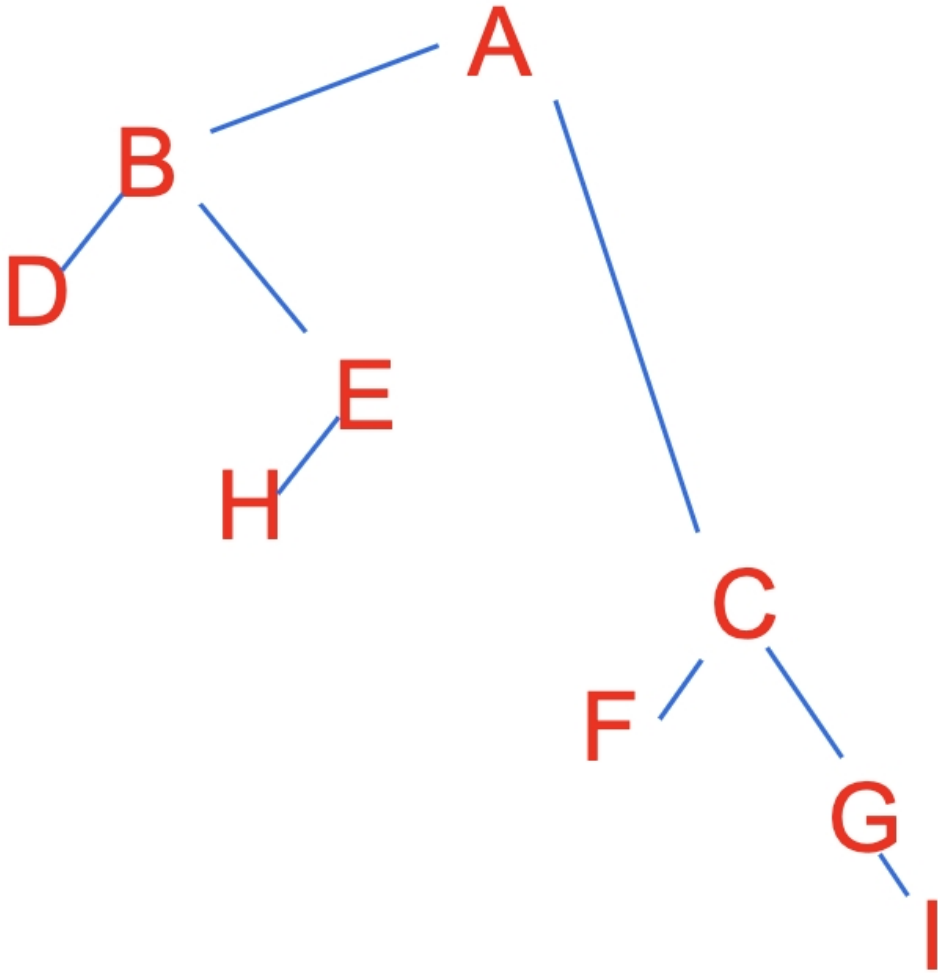
后续遍历倒过来
相同的对齐。

前序遍历

A
B
D
E
H
C
F
G
I

中序遍历

D B H E A F C G I



不确定的还是要验证！

已知先序和后序可以确定一颗树吗？

前序：ABDEHCFG I

后序：DHEBFIGCA

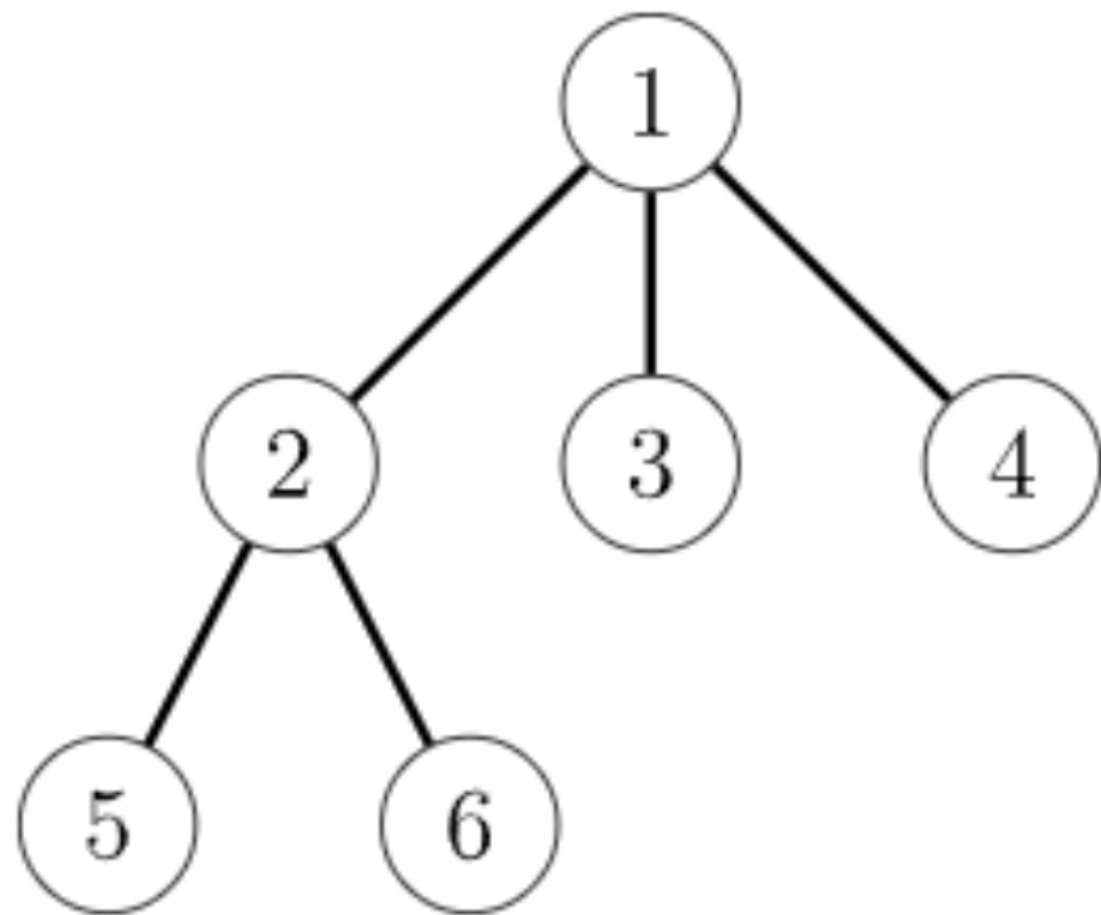
- 先序遍历：A B
- 后序遍历：B A



若某节点只有一个子节点，则无法确定是左还是右

层序BFS（广度优先）遍历

树层序遍历是指按照按**从上到下、从左到右的顺序逐层**访问二叉树的节点。层序遍历要求将不同的层次区分开来，所以其结果通常以**二维数组**的形式表示。



层序遍历: **[[1],[2,3,4],[5,6]]**

练习

1.已知前序和中序，画出二叉树

前序：1 2 4 3 5 7 6

中序：2 4 1 7 5 3 6

2.已知后序和中序，求前序

后序：DGJHEBIFCA

中序：DBGEHJACIF

选项：

A. ABCDEFGHIJ

B. ABDEGHJCFI

C. ABDEGJHCFI

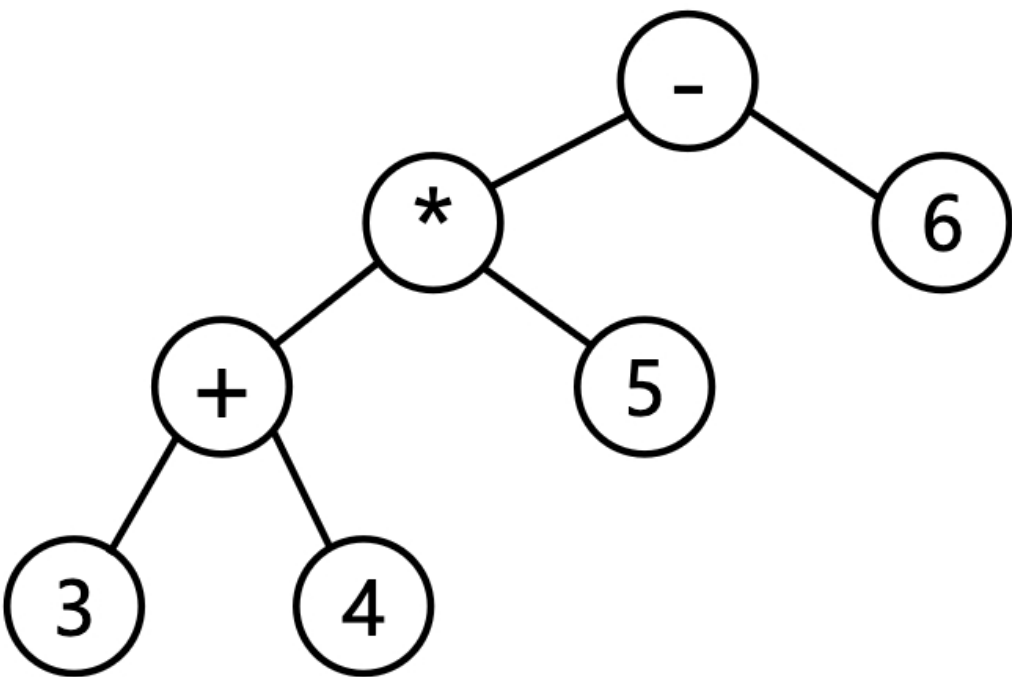
D. ABDEGHJFIC

答案：



波兰式——算术表达式的前中后缀表达式

表达式	定义	示例
前缀表达式	运算符位于操作数之前	- × + 3 4 5 6
中缀表达式	操作符以中缀形式处于操作数的中间	(3 + 4) × 5 - 6
后缀表达式	运算符位于操作数之后	3 4 + 5 × 6 -



- 先序遍历 前缀表达式（波兰式）
- 中序遍历 中缀表达式
- 后序遍历 后缀表达式（逆波兰式）

前缀、中缀、后缀表达式互相转换

中缀转前缀

- 1.初始化两个栈：运算符栈S1和储存中间结果的栈S2；**从右至左**扫描中缀表达式；遇到操作数时，将其压入S2；遇到运算符时，比较其与S1栈顶运算符的优先级：
 - 2.如果S1为空，或栈顶运算符为右括号“)” ，则直接将此运算符入栈； 否则，若优先级比栈顶运算符的**较高或相等**，也将运算符压入S1；
 - 3.否则，将S1栈顶的运算符弹出并压入到S2中，再次与S1中新的栈顶运算符相比较
- 遇到括号时：
- 4.如果是右括号“)” ，则直接压入S1；
 - 5.如果是左括号“(“ ，则依次弹出S1栈顶的运算符，并压入S2，直到遇到右括号为止，此时将这一对括号丢弃
- 重复步骤2至5，直到表达式的最左边；
- 将S1中剩余的运算符依次弹出并压入S2；
- 依次弹出S2中的元素并输出，结果即为中缀表达式对应的前缀表达式。

前缀表达式**逆向**求解 中缀表达式

$- + 1 * + 2 3 4 5$

思路：递归，碰到操作符就进入递归。

前缀、中缀、后缀表达式互相转换

中缀转后缀

- 1.初始化两个栈：运算符栈S1和储存中间结果的栈S2；**从左至右**扫描中缀表达式；遇到操作数时，将其压入S2；遇到运算符时，比较其与S1栈顶运算符的优先级：
 - 2.如果S1为空，或栈顶运算符为左括号“（”，则直接将此运算符入栈；比栈顶高，也将运算符压入S1 **（注意转换为前缀表达式时是优先级较高或相同，而这里则不包括相同的情况）**；
 - 3.比栈顶低或相同，将S1栈顶的运算符弹出并压入到S2中，再与S1中新的栈顶运算符相比较；
- 遇到括号时：
- 4.如果是左括号“（”，则直接压入S1；
 - 5.如果是右括号“）”，则依次弹出S1栈顶的运算符，并压入S2，直到遇到左括号为止，此时将这一对括号丢弃；
- 可以想象成“（”比任何运算符都高，“）”比任何运算符都低。**
- 重复步骤2至5，直到表达式的最右边；
- 将S1中剩余的运算符依次弹出并压入S2；
- 依次弹出S2中的元素并输出，结果的逆序即为中缀表达式对应的后缀表达式 **（转换为前缀表达式时不用逆序）**。

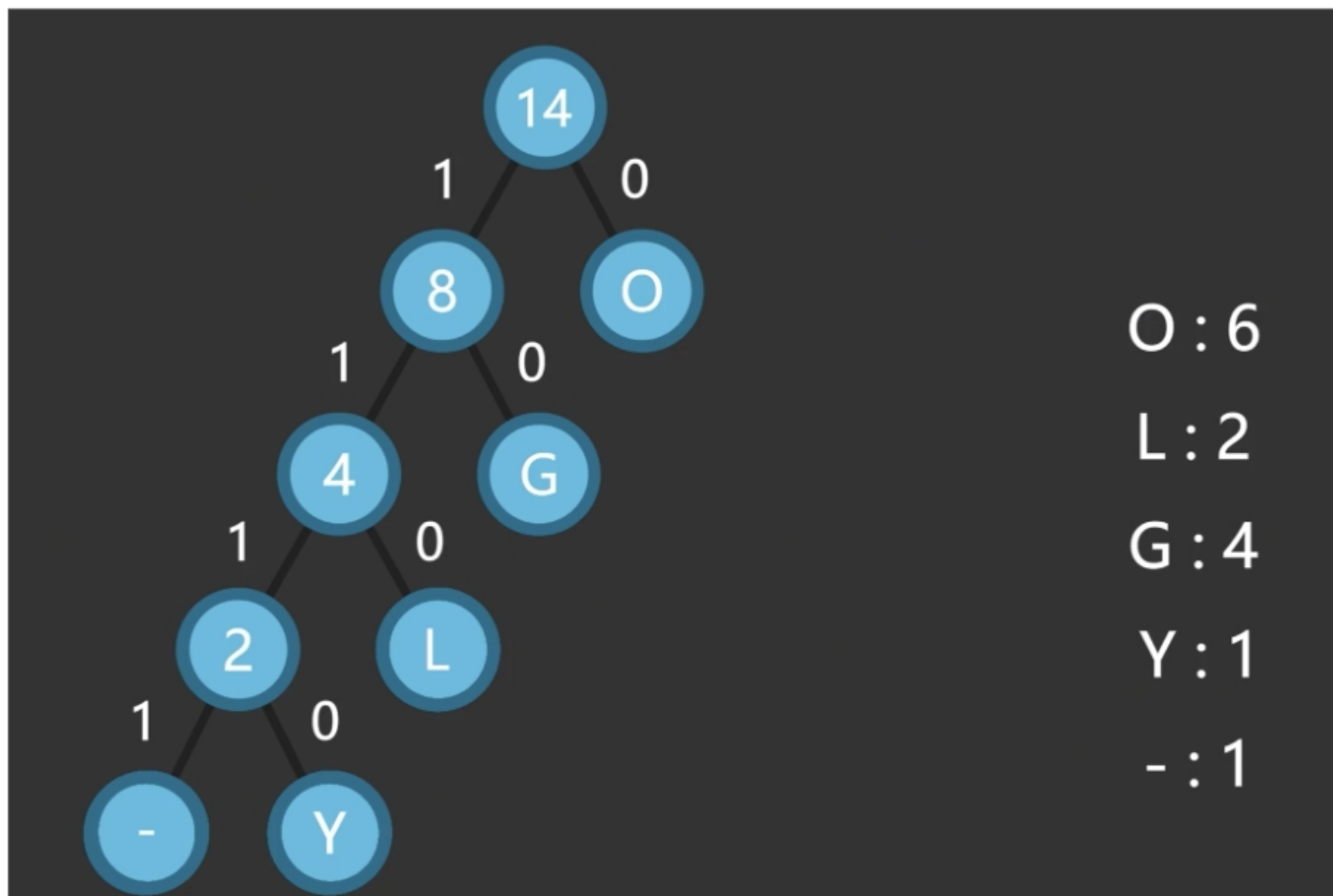
后缀表达式 逆向求解 中缀表达式

1 2 3 + 4 * 5 - +

基本思路和上面的一样：递归，碰到操作符就进入递归

哈夫曼编码-哈夫曼树

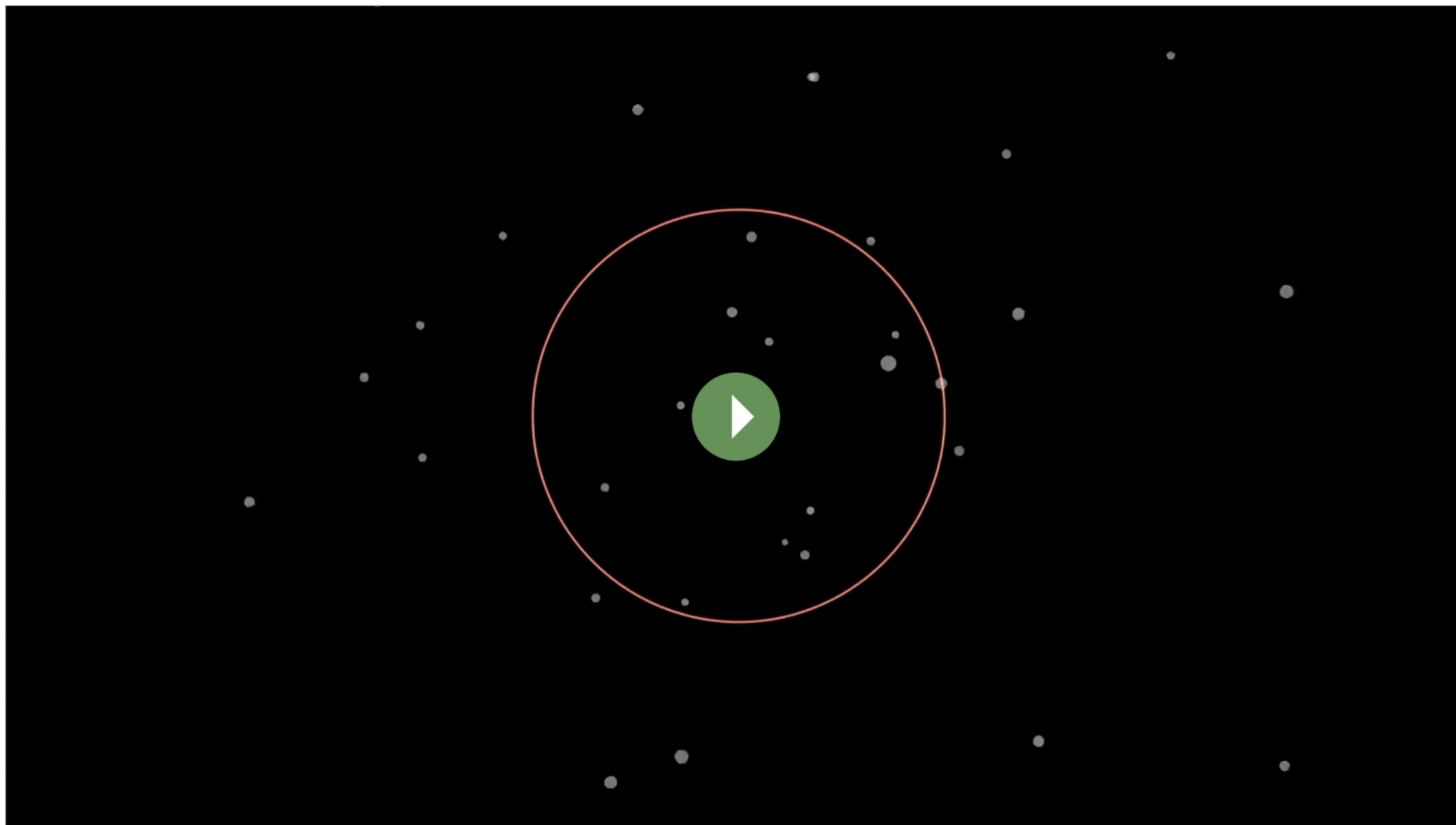
哈夫曼树 (Huffman Tree) 是一种用于数据压缩的二叉树。它基于字符出现的频率构建，使得高频字符使用较短的编码，低频字符使用较长的编码，从而实现数据压缩。哈夫曼树也被称为最优二叉树或哈夫曼编码树。



哈夫曼树的构建过程

1. 统计频率: 统计每个字符在数据中出现的频率。
2. 构建优先队列: 将每个字符及其频率作为一个节点, 并按照频率从小到大的顺序放入优先队列 (最小堆)。
3. 构建哈夫曼树:
从优先队列中取出两个频率最小的节点, 构建一个新的父节点, 其频率为两个子节点频率之和。将新的父节点插回优先队列。重复上述步骤, 直到队列中只剩下一个节点, 即哈夫曼树的根节点。
4. 生成编码: 从根节点出发, 对每个节点规定左边路径为0, 右边路径为1, 最终从根节点到叶节点的路径构成了该字符的哈夫曼编码。

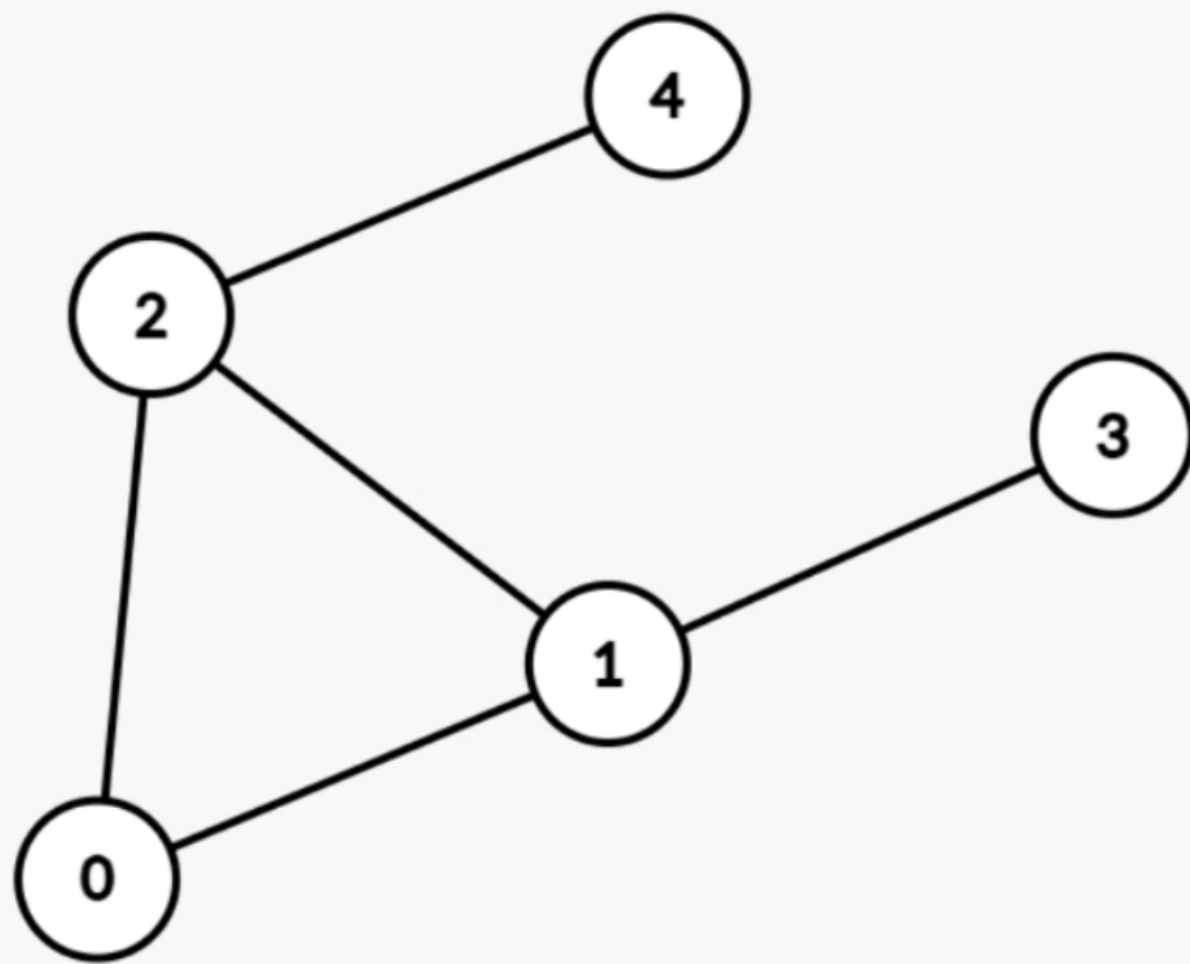
哈夫曼编码



图

图 (graph)是一个二元组 $G=(V(G),E(G))$ 。其中 $V(G)$ 是非空集，称为**点集 (vertex set)**，对于 V 中的每个元素，我们称其为**顶点 (vertex)**或**节点 (node)**，简称**点**； $E(G)$ 为 $V(G)$ 各结点之间边的集合，称为**边集 (edge set)**。

常用 $G=(V, E)$ 表示图。



无向图

图的相关概念

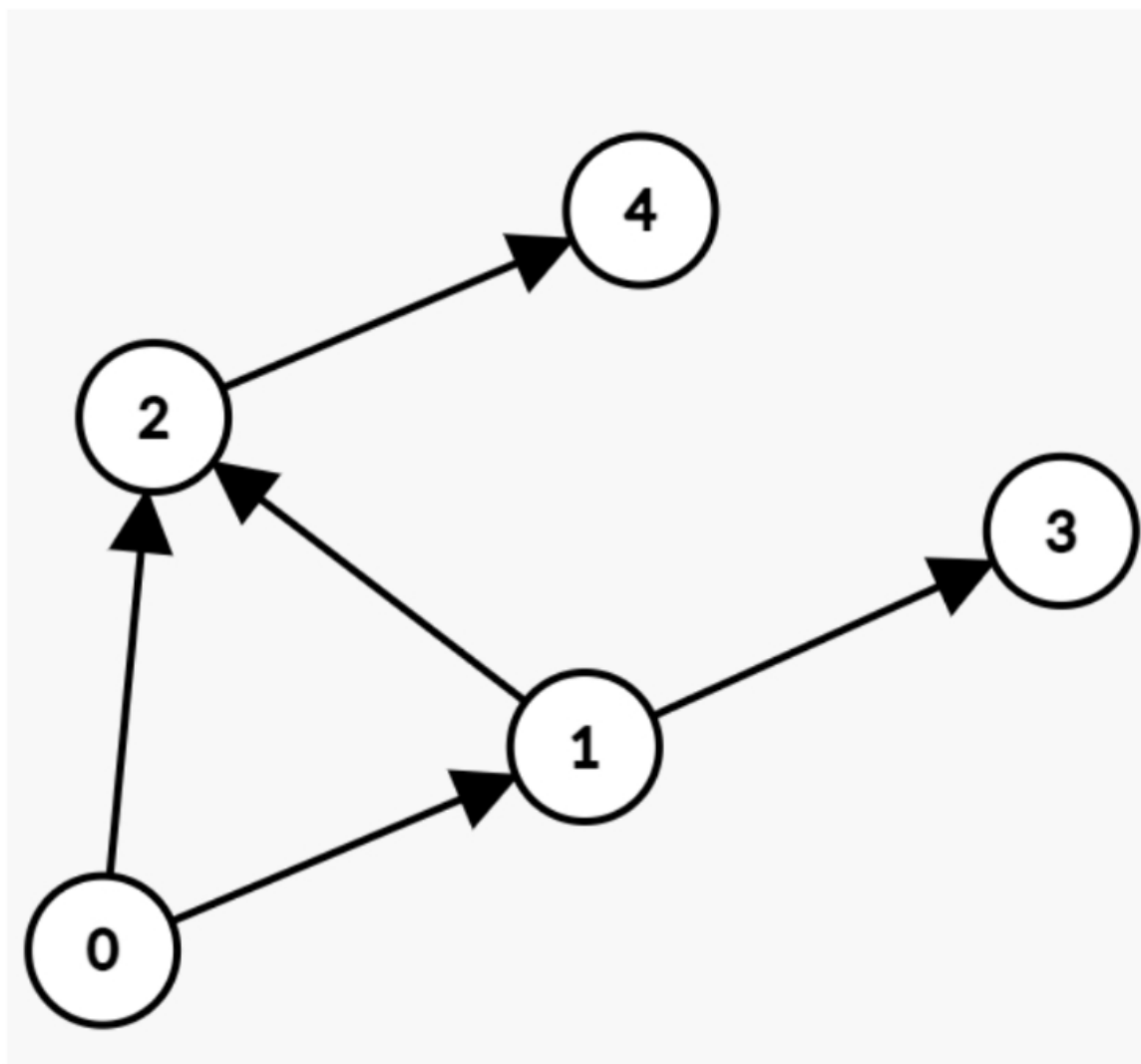
当 V, E 都是有限集合时，称 G 为**有限图**。

当 V 或 E 是无限集合时，称 G 为**无限图**。

图有多种，包括**无向图 (undirected graph)**，**有向图 (directed graph)**，**混合图 (mixed graph)**等。

若 G 为无向图，则 E 中的每个元素为一个无序二元组 (u, v) ，称作**无向边 (undirected edge)**，简称**边 (edge)**，则 u 和 v 称为边的**端点 (endpoint)**。

若 G 为有向图，则 E 中的每一个元素为一个有序二元组 (u, v) ，有时也写作 $u \rightarrow v$ ，称作**有向边 (directed edge)**或**弧 (arc)**，在不引起混淆的情况下也可以称作**边 (edge)**。则此时 u 称为边的**起点 (tail)**， v 称为边的**终点 (head)**。

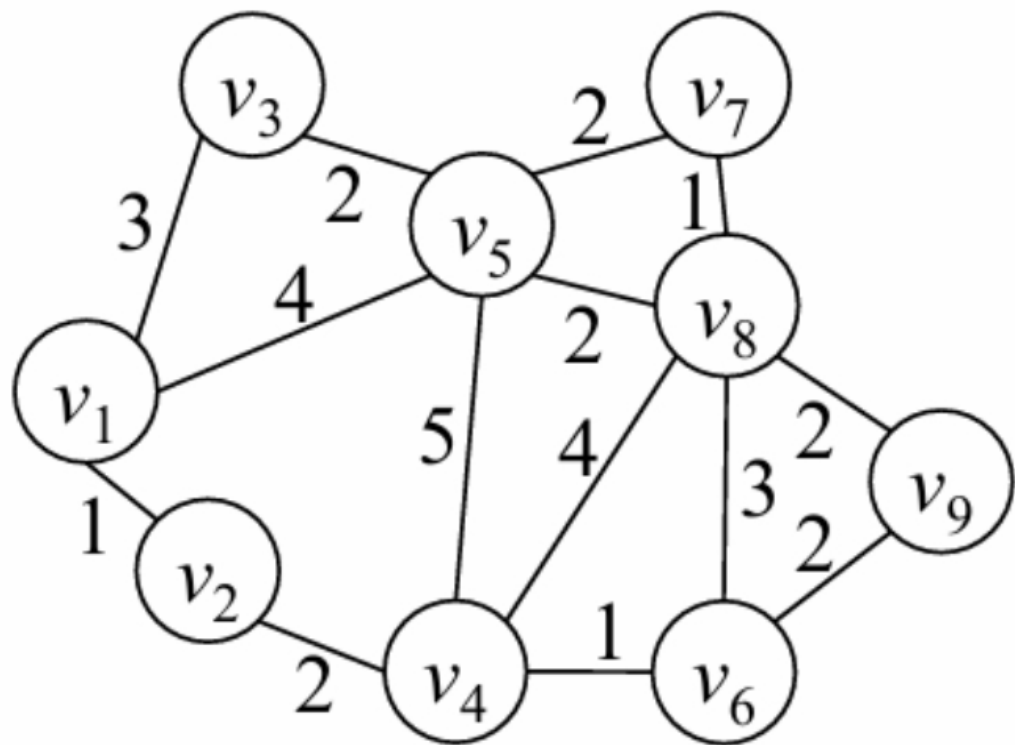


有向图

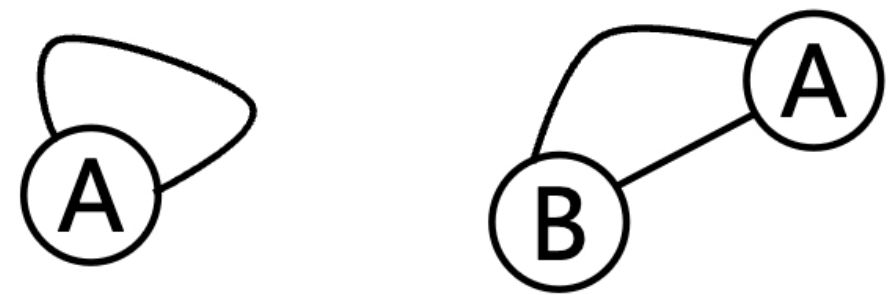
若图G的每条边都被赋予一个数作为该边的**权**，则称G为**赋权图**。如果这些权都是正实数，就称G为**正权图**。

图G的点数 $|V(G)|$ 也被称作图G的**阶 (order)**。

在无向图 $G=(V,E)$ 中，若点v是边e的一个端点，则称v和e是**关联的 (incident)**或**相邻的 (adjacent)**。对于两顶点u和v，若存在边 (u,v) ，则称u和v是**相邻的 (adjacent)**。



- 自环 (loop)**: 对边 $e=(u,v)$ ，若 $u=v$ ，则e被称作一个自环。
- 重边 (multiple edge)**: 若E中存在两个完全相同的元素（边） e_1, e_2 则它们被称作（一组）重边。
- 简单图 (simple graph)**: 若一个图中没有自环和重边，它被称为简单图。**具有至少两个顶点的简单无向图中一定存在度相同的结点。**
- 如果一张图中有自环或重边，则称它为**多重图 (multigraph)**。



度数

与一个顶点 v 关联的边的条数称作该顶点的**度 (degree)**，记作 $d(v)$ 。

握手定理 (又称图论基本定理)：对于任何无向图 $G=(V,E)$ ，有 $\sum_{v \in V} d(v) = 2|E|$ 。

即无向图的边数等于各顶点度数之和的一半。

推论：在任意图中，度数为奇数的点必然有偶数个。

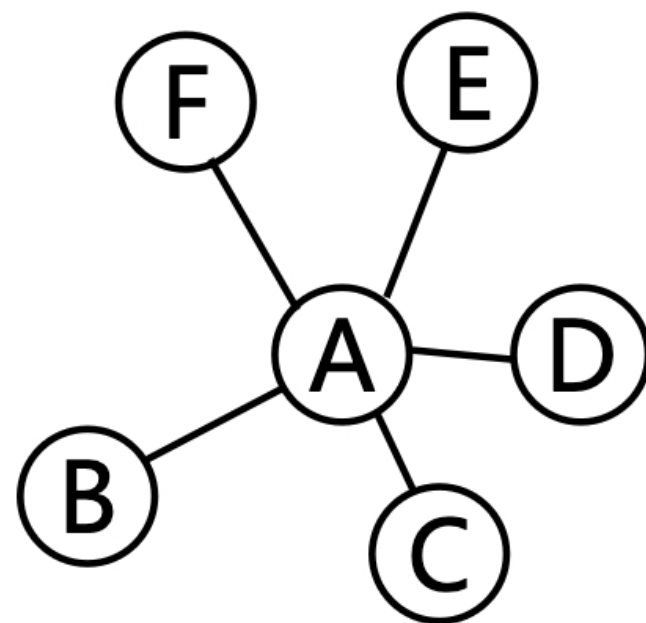
若度为0，则称 v 为**孤立点 (isolated vertex)**。

若度为1，则称 v 为**叶节点 (leaf vertex)/悬挂点 (pendant vertex)**。

若 $2 \mid d(v)$ ，则称 v 为**偶点 (even vertex)**。

若 $2 \nmid d(v)$ ，则称 v 为**奇点 (odd vertex)**。图中奇点的个数是偶数。

若 $d(v) = |V| - 1$ ，则称 v 为**支配点 (universal vertex)**。



A: 支配点

在有向图 $G=(V,E)$ 中，以一个顶点 v 为起点的边的条数称为该顶点的**出度 (out-degree)**，记作 $d^+(v)$ 。以一个顶点 v 为终点的边的条数称为该节点的**入度 (in-degree)**，记作 $d^-(v)$ 。显然 $d^+(v)+d^-(v)=d(v)$ 。

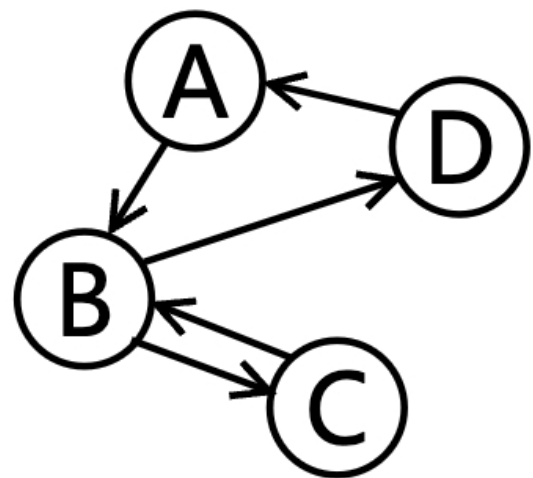
对于任意的有向图，所有点的出度和=所有点的入度和=边数

迹 (trail)：对于一条途径 w ，若**边两两互不相同**，则称 w 是一条迹。

路径 (path)（又称**简单路径 (simple path)**）：对于一条迹 w ，若其连接的点的序列中点**两两不同**，则称 w 是一条路径。

回路 (circuit)：对于一条迹 w ，若首点和最后一个点相同，则称 w 是一条回路。

环/圈 (cycle)（又称**简单回路/简单环 (simple circuit)**）：对于一条回路 w ，若首点和最后一个点是点序列中唯一重复出现的点对，则称 w 是一个环。

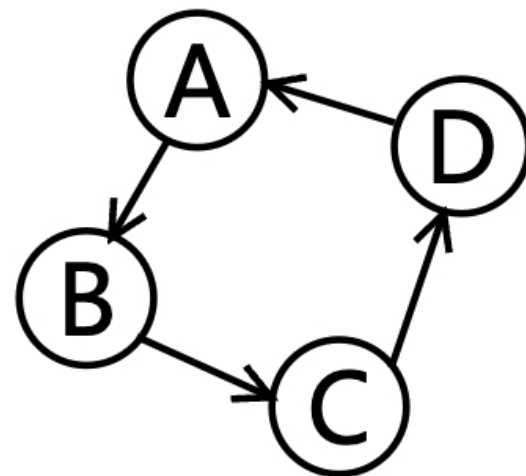


回路

A-B-C-B-D-A

A-B-C-D-A

环

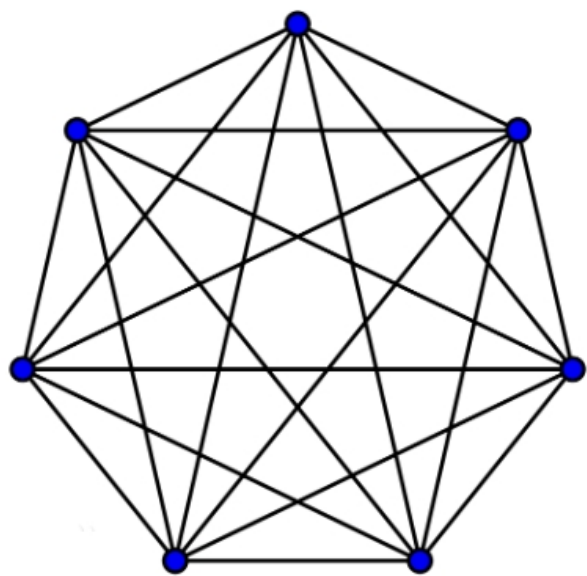


图的连通

若无向图 $G=(V,E)$ ，满足其中任意两个顶点均连通，则称 G 是**连通图 (connected graph)**， G 的这一性质称作**连通性 (connectivity)**。

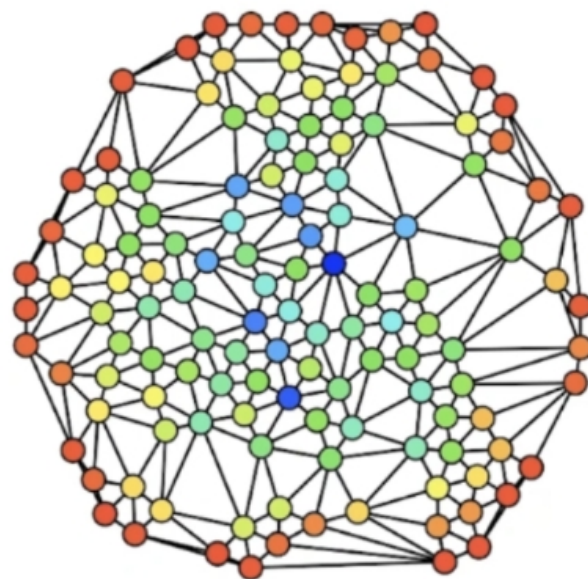
完全图是一个简单的无向图，其中**每对不同的顶点之间都恰连有一条边**相连。

n 个端点的完全图有 n 个端点以及 $n(n - 1) / 2$ 条边



完全图

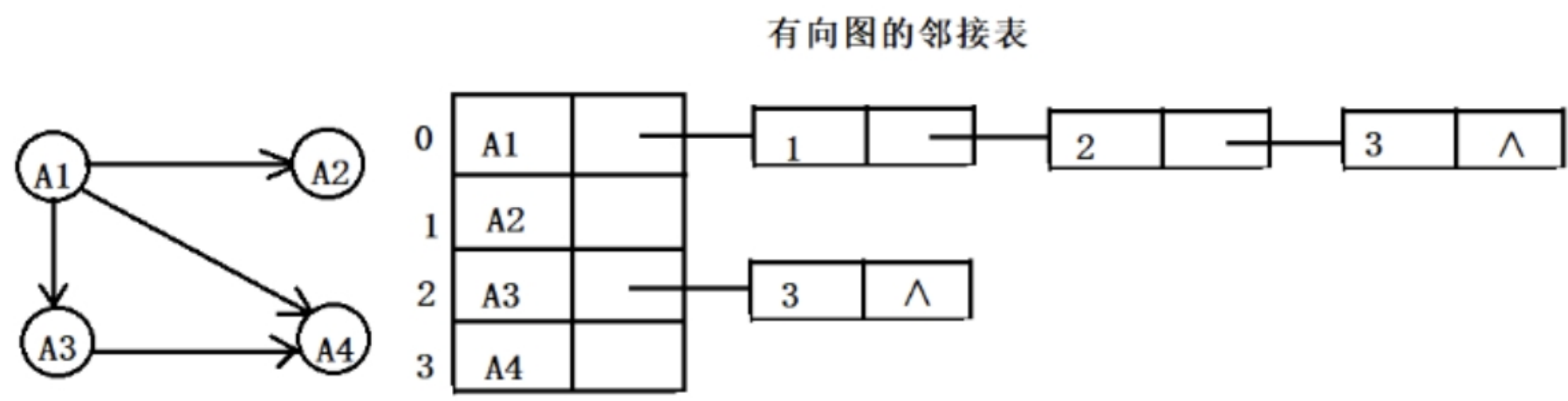
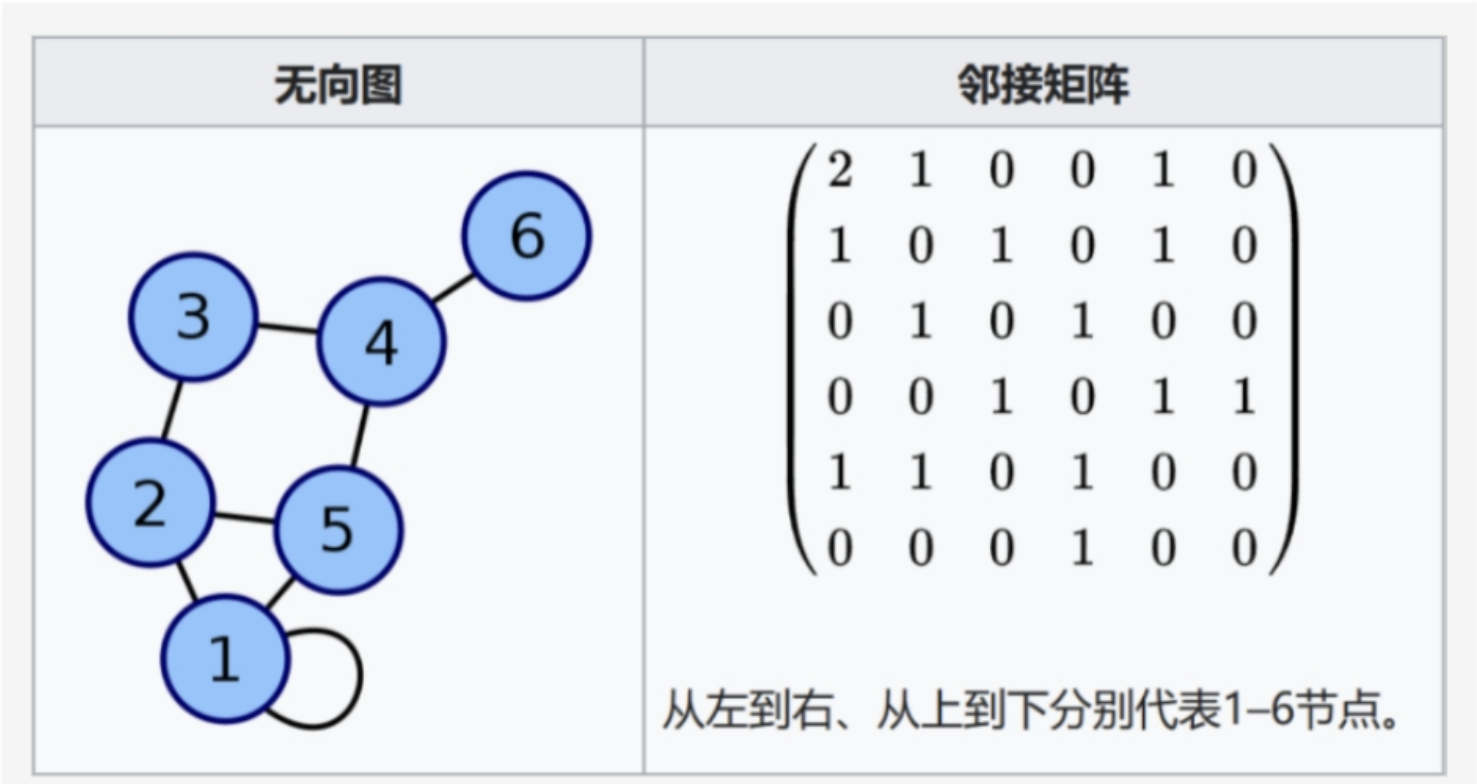
稀疏图



稠密图



图的存储



拓扑排序

对于有向无环图，按照先后顺序排列所有顶点。

方法：统计入度，将入度为0的点入队，依次弹出并删除其出边。

若最终不能排出所有点，则图中有环。

